Advanced Methods in Deep Learning

Lecture 3: Training Neural Networks – SGD, Regularization, Generalization

Alexander Quispe Rojas
Universidad del Pacífico – MECA
aw.quispero@up.edu.pe aquisper@caltech.edu

April 20, 2026

Today's Roadmap

Recap and Training Challenges

Advanced Optimizers

Weight Initialization

Regularization Techniques

Batch Normalization

Universal Approximation Theorem

GPUs and Parallel Computing

Practical Recipe and Summary

Recap & Training Challenges

Recap from Lecture 2

What we learned:

- **Backpropagation**: systematic application of the chain rule

$$\boldsymbol{\delta}^{(l)} = [(\mathbf{W}^{(l+1)})^\top \boldsymbol{\delta}^{(l+1)}] \odot \sigma'(\mathbf{z}^{(l)})$$

- **SGD** with mini-batches:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\mathbf{w}} \ell_i$$

- **Momentum** smooths the trajectory: $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \nabla \mathcal{L}$.

Today's question:

How do we actually train deep networks **in practice**?

What Can Go Wrong During Training?

Symptom	Likely cause	Fix (covered today)
Loss doesn't decrease	Wrong LR, bad initialization	Better optimizer, init
Loss oscillates wildly	LR too high, batch too small	Adaptive LR (Adam)
Train loss $\downarrow$ but val loss $\uparrow$	Overfitting	Regularization, dropout
Gradients vanish/explode	Deep network, bad init	Batch norm, He init
Very slow convergence	Poor landscape	Momentum, Adam

Today's toolkit: Advanced optimizers (Adam) · Weight initialization (Xavier, He) · Regularization (L1, L2, dropout) · Batch normalization · Universal Approximation Theorem · GPUs and parallel computing.

Advanced Optimizers

The Problem with Plain SGD

Plain SGD uses the **same learning rate** for all parameters:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} \mathcal{L}$$

But some parameters need different treatment:

- Parameters with **large** gradients: small LR needed (avoid overshooting).
- Parameters with **small** gradients: large LR needed (make progress).
- **Sparse features**: rarely updated, need larger steps when they appear.

Solution: adaptive learning rates — each parameter gets its own step size based on its gradient history.

AdaGrad: Adaptive Gradients

AdaGrad (Duchi et al., 2011)

Accumulate squared gradients: $G_t = G_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t$ where $\mathbf{g}_t = \nabla \mathcal{L}(\mathbf{w}_t)$.
Update rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

Per-parameter learning rate: $\alpha_i = \frac{\alpha}{\sqrt{G_{t,i}+\epsilon}}$.

Intuition:

- Parameters with large cumulative gradients $\rightarrow$ smaller LR.
- Parameters with small cumulative gradients $\rightarrow$ larger LR.

Problem: G_t grows monotonically $\Rightarrow$ LR decays to zero $\Rightarrow$ training stalls.

RMSprop: Exponential Moving Average

Fix for AdaGrad: use an **exponential moving average** instead of cumulative sum:

RMSprop (Hinton, 2012)

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1-\rho)\mathbf{g}_t \odot \mathbf{g}_t$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\alpha}{\sqrt{\mathbf{v}_t + \epsilon}} \odot \mathbf{g}_t$$

where $\rho \approx 0.9$ (typical).

Key insight: $\mathbf{v}_t$ is a weighted average that **forgets** old gradients, so the LR doesn't decay to zero.

Problem solved: AdaGrad's aggressive LR decay. But still missing momentum.

Adam: Momentum + RMSprop

Adam combines momentum and RMSprop — the current standard optimizer.

Moment estimates:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \qquad \text{(first moment, like momentum)}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t \odot \mathbf{g}_t \qquad \text{(second moment, like RMSprop)}$$

Bias-corrected: $\hat{\mathbf{m}}_t = \dfrac{\mathbf{m}_t}{1-\beta_1^t} \qquad \hat{\mathbf{v}}_t = \dfrac{\mathbf{v}_t}{1-\beta_2^t}$

Update: $\mathbf{w}_{t+1} = \mathbf{w}_t - \dfrac{\alpha\, \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}$

Defaults: $\alpha = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Why Adam Works So Well

Adam combines three ideas:

- **Momentum** ($\mathbf{m}_t$): smooth the gradient direction, help with noise.
- **Adaptive LR** ($\mathbf{v}_t$): each parameter gets its own step size.
- **Bias correction**: crucial for the first few iterations.

Why bias correction? At $t = 1$, if $\mathbf{m}_0 = \mathbf{0}$:

$$\mathbf{m}_1 = (1 - \beta_1)\mathbf{g}_1 \approx 0.1\,\mathbf{g}_1$$

So $\mathbf{m}_1$ is **biased toward zero**. Dividing by $1 - \beta_1^t$ corrects this:

$$\hat{\mathbf{m}}_1 = \frac{(1 - \beta_1)\mathbf{g}_1}{1 - \beta_1} = \mathbf{g}_1$$

Practical advice:

- Start with **Adam**, LR = 10^{-3}.
- Switch to **SGD + momentum** for final fine-tuning if needed.

Optimizer Comparison

Optimizer	Update rule (schematic)	Hyperparams	Use case
SGD	$\mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{g}$	α	Final fine-tune
SGD + momentum	$\mathbf{v} \leftarrow \beta \mathbf{v} + \mathbf{g}$; $\mathbf{w} \leftarrow \mathbf{w} - \alpha \mathbf{v}$	α, β	Computer vision
AdaGrad	Cumulative $\mathbf{g}^2$ scaling	α	Sparse / NLP
RMSprop	EMA of $\mathbf{g}^2$	α, ρ	RNNs
Adam	Momentum + RMSprop + bias corr.	α, β_1, β_2	Default
AdamW	Adam + decoupled weight decay	$\alpha, \beta_1, \beta_2, \lambda$	Transformers

Rule of thumb: When in doubt, use Adam with default parameters.

Weight Initialization

Why Initialization Matters

Bad initialization can kill training. Consider a few options:

Zero init ($\mathbf{W} = \mathbf{0}$):

- All neurons in a layer compute the same thing.
- Gradients are identical $\Rightarrow$ symmetry never breaks.
- Network cannot learn anything useful.

Too small ($\mathbf{W} \sim \mathcal{N}(0, 0.01^2)$):

- Activations and gradients shrink through layers.
- **Vanishing gradient** problem.

Too large ($\mathbf{W} \sim \mathcal{N}(0, 1^2)$ for large layers):

- Activations grow through layers.
- **Exploding gradient** problem.

Goal: Keep variance of activations roughly constant across layers.

Xavier / Glorot Initialization

Consider a linear layer: $z = \sum_{i=1}^{n_{in}} W_i x_i$ with $\mathbb{E}[x_i] = 0$, $\mathrm{Var}(x_i) = \sigma_x^2$.
Assuming W_i independent with $\mathbb{E}[W_i] = 0$, $\mathrm{Var}(W_i) = \sigma_W^2$:

$$\mathrm{Var}(z) = \sum_{i=1}^{n_{in}} \mathrm{Var}(W_i x_i) = n_{in} \cdot \sigma_W^2 \cdot \sigma_x^2$$

To preserve variance ($\mathrm{Var}(z) = \mathrm{Var}(x)$): $\sigma_W^2 = \dfrac{1}{n_{in}}$

Xavier/Glorot (2010)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

Designed for **tanh** / **sigmoid** activations. Uses average of n_{in}, n_{out} for balance.

He Initialization (for ReLU)

Problem with Xavier + ReLU: ReLU zeroes out half the neurons, so variance halves at each layer.

Fix: double the variance to compensate.

He Initialization (2015)

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

Designed for **ReLU** activations. Used in ResNet, most modern CNNs.

Activation	Initialization	Variance
Sigmoid / Tanh	Xavier / Glorot	$2/(n_{in} + n_{out})$
ReLU / Leaky ReLU	He / Kaiming	$2/n_{in}$

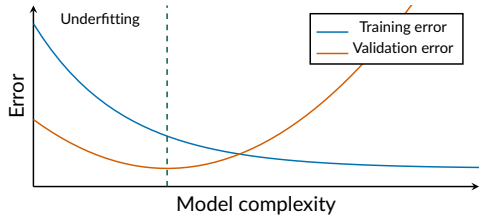
Good news: PyTorch uses sensible defaults; set explicitly for custom architectures.

Regularization

Why Regularize?

Deep networks can easily overfit: millions of parameters, finite training data.

Bias-variance tradeoff:



Regularization: techniques that reduce overfitting by **constraining** the model.

Today we cover: L2, L1, dropout, early stopping, data augmentation.

L2 Regularization (Weight Decay)

Add a penalty for large weights to the loss:

L2 Regularization

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 = \mathcal{L}(\mathbf{w}) + \frac{\lambda}{2}\sum_i w_i^2$$

where $\lambda > 0$ is the regularization strength.

Gradient: $\nabla_{\mathbf{w}}\tilde{\mathcal{L}} = \nabla_{\mathbf{w}}\mathcal{L} + \lambda\mathbf{w}$.
Update rule:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha(\nabla_{\mathbf{w}}\mathcal{L} + \lambda\mathbf{w}) = (1 - \alpha\lambda)\mathbf{w} - \alpha\nabla_{\mathbf{w}}\mathcal{L}$$

This is why L2 is called "weight decay": weights shrink by a factor $(1 - \alpha\lambda)$ each step.

Bayesian interpretation: L2 corresponds to a **Gaussian prior** $\mathbf{w} \sim \mathcal{N}(0, 1/\lambda)$ on the weights.

L1 Regularization (Sparsity)

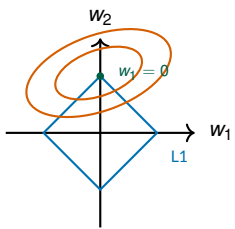
L1 Regularization

$$\tilde{\mathcal{L}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 = \mathcal{L}(\mathbf{w}) + \lambda \sum_i |w_i|$$

Key property: L1 induces **sparsity** — many weights become exactly zero.

Why sparsity?
Geometric intuition: L1 constraint $|w_1| + |w_2| \leq c$ is a **diamond**. The optimum typically lies at a **corner** (axis), where some $w_i = 0$.

Bayesian interpretation: L1 = **Laplace prior** on the weights.



Dropout

Dropout (Srivastava et al., 2014)

During training, randomly set a fraction p of neurons to zero at each forward pass:

$$\mathbf{h}_{\text{drop}} = \mathbf{h} \odot \mathbf{m}, \qquad m_i \sim \text{Bernoulli}(1-p) \text{ independently}$$

At inference time: use all neurons, but **scale** by $(1-p)$ to match expected output during training.

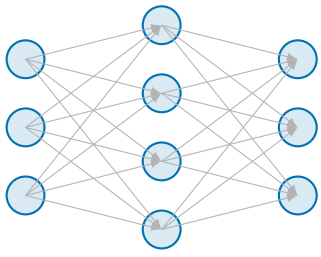
Why dropout works:

- **Prevents co-adaptation**: neurons can't rely on specific others.
- **Ensemble interpretation**: each forward pass uses a different sub-network.
- **Implicit regularization**: similar effect to L2.

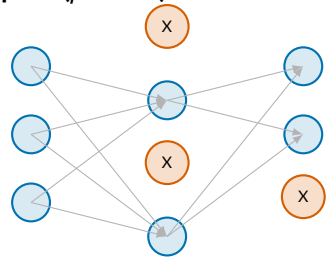
Typical values: $p = 0.2$ for input layer, $p = 0.5$ for hidden layers.

Dropout: Visual Illustration

Without dropout



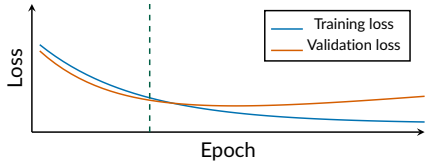
With dropout ($p = 0.5$)



At each training step, a different random subset of neurons is active.

Early Stopping and Data Augmentation

Early stopping: monitor validation loss and stop when it stops improving.



Data augmentation: create “new” training data via transformations.

- **Images:** rotation, flipping, cropping, color jittering.
- **Text:** synonym replacement, back-translation.
- **Time series:** jittering, time warping.
- **Effect:** teach the model **invariances**.

Batch Normalization

The Internal Covariate Shift Problem

In a deep network, the distribution of activations at each layer **changes** during training (as earlier layers' weights change).

Consequences:

- Each layer must constantly adapt to its input distribution.
- Training is slower.
- Requires careful initialization and small learning rates.

Idea (Ioffe & Szegedy, 2015): normalize activations at each layer to have mean 0 and variance 1.

But naive normalization destroys information. The solution: normalize, then learn scale and shift.

Batch Normalization: The Algorithm

For a mini-batch $\mathcal{B} = \{x_1, \ldots, x_B\}$:

Step 1 – Compute batch statistics:

$$\mu_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B} x_i \qquad \sigma^2_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B}(x_i - \mu_{\mathcal{B}})^2$$

Step 2 – Normalize: $\hat{x}_i = \dfrac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}$

Step 3 – Scale and shift (learnable): $y_i = \gamma \hat{x}_i + \beta$

where γ, β are **learned parameters** (via backprop).

Key insight: if the network learns $\gamma = \sigma_{\mathcal{B}}, \beta = \mu_{\mathcal{B}}$, BN is identity $\Rightarrow$ no loss of expressiveness.

Batch Norm: Training vs Inference

At training time: use **batch statistics** $\mu_{\mathcal{B}}, \sigma^2_{\mathcal{B}}$.

At inference time: batches may have size 1, so batch statistics are unreliable.
Solution: maintain **running averages** during training:

$$\mu_{\text{run}} \leftarrow (1-m)\mu_{\text{run}} + m\mu_{\mathcal{B}}$$
$$\sigma^2_{\text{run}} \leftarrow (1-m)\sigma^2_{\text{run}} + m\sigma^2_{\mathcal{B}}$$

At inference, use $\mu_{\text{run}}, \sigma^2_{\text{run}}$.

Benefits of BN:

- Faster training (allows higher LR).
- Smoother loss landscape (shown empirically).
- Implicit regularization (batch noise acts as regularization).
- Less sensitive to initialization.

Variants: Layer Norm (Transformers), Group Norm, Instance Norm.

Universal Approximation Theorem

The Universal Approximation Theorem

Cybenko (1989), Hornik (1991)

Let σ be a non-constant, bounded, monotonically increasing continuous function. Then for any continuous function $f : [0,1]^d \to \mathbb{R}$ and any $\epsilon > 0$, there exist $N \in \mathbb{N}$, $\{w_i\}, \{b_i\}, \{v_i\}$ such that:

$$\left| f(\mathbf{x}) - \sum_{i=1}^{N} v_i \, \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i) \right| < \epsilon \quad \forall \mathbf{x} \in [0,1]^d$$

In words: a neural network with **one hidden layer** and **enough neurons** can approximate any continuous function to arbitrary precision.

Extensions:

- Works for sigmoid, tanh, ReLU (with adjustments).
- Also holds for any Lebesgue-measurable function.

UAT: Proof Sketch (Geometric Intuition)

Goal: approximate any function $f(x)$ with a sum of sigmoids.

Step 1: A single sigmoid can approximate a **step function** (with large w):

$$\sigma(wx+b) \approx \begin{cases} 1 & x > -b/w \\ 0 & x < -b/w \end{cases} \quad \text{for large } w$$

Step 2: Two sigmoids with opposite weights create a **bump** function:

$$\sigma(w(x-a)) - \sigma(w(x-a-h)) \approx \text{bump on } [a, a+h]$$

Step 3: Sum many bumps $\rightarrow$ approximate any continuous function (like Riemann integration!).

Formal argument: uses the Hahn-Banach theorem + Riesz representation theorem.

Key insight: sigmoids form a dense subset of continuous functions on compact sets.

But Why Depth Then?

UAT says: one hidden layer is enough. But in practice, we use **deep** networks.

Problem with shallow networks: the required width can grow **exponentially** with complexity.

Theorem

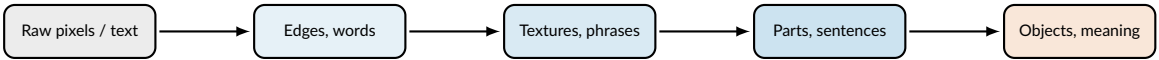
Depth-width tradeoff (Telgarsky, 2016):
There exist functions that can be represented by a network of depth L with $O(L)$ neurons, but require $\Omega(2^L)$ neurons in a shallow network.

Why deep works:

- **Hierarchical features**: edges $\rightarrow$ parts $\rightarrow$ objects.
- **Compositional structure**: natural data is compositional.
- **Exponential efficiency**: depth is **exponentially more parameter-efficient**.

Why Depth Matters – Economic Analogy

Deep networks learn hierarchical representations:



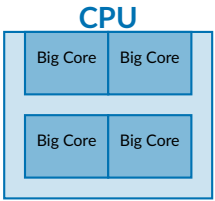
Analogy with economics:

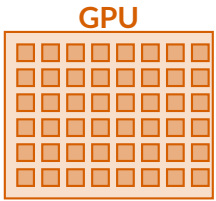
- Raw data: prices, quantities, transactions.
- Layer 1: growth rates, ratios, simple features.
- Layer 2: trends, seasonality, co-movements.
- Layer 3: regime indicators, crisis flags.
- Output: GDP forecast, policy recommendation.

Economic relationships are often compositional — which is why depth helps.

GPUs and Parallel Computing

CPU vs GPU Architecture





- Few powerful cores (4–64).
- Optimized for **sequential** execution.
- Great for branching logic.

- Thousands of simple cores.
- Optimized for **parallel** execution.
- Great for matrix operations.

Why GPUs dominate DL: neural networks are mostly **matrix multiplications**, which are embarrassingly parallel. A 1000×1000 matmul: CPU = seconds, GPU = milliseconds.

Using GPUs in PyTorch

```python
import torch

# Check if GPU is available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Move tensors to GPU
x = torch.randn(1000, 1000).to(device)
y = torch.randn(1000, 1000).to(device)
z = x @ y                       # Matmul on GPU

# Move model to GPU
model = MyNeuralNetwork().to(device)

# Move batches to GPU during training
for X_batch, y_batch in loader:
    X_batch, y_batch = X_batch.to(device), y_batch.to(device)
    y_pred = model(X_batch)
    loss = criterion(y_pred, y_batch)
```

Rule: both model and data must be on the **same device**.

Parallelism Strategies

Data parallelism (most common):

- Replicate model across multiple GPUs.
- Split batch across GPUs.
- Average gradients after each step.
- PyTorch: `nn.DataParallel`, `nn.DistributedDataParallel`.

Model parallelism (for huge models):

- Split model across multiple GPUs.
- Used for LLMs that don't fit in one GPU.

Mixed precision training (AMP):

- Use float16 instead of float32 where possible.
- **2x faster** on modern GPUs (tensor cores).
- PyTorch: `torch.cuda.amp.autocast`.

For this course: Google Colab GPUs are enough (T4, A100).

Practical Recipe & Summary

The Deep Learning Training Recipe

A checklist for training deep networks:

1. **Normalize your inputs** (mean 0, std 1 per feature).
2. **Initialize weights properly:** He (ReLU) or Xavier (sigmoid/tanh).
3. **Choose architecture:** start simple, add complexity as needed.
4. **Add batch normalization** to hidden layers.
5. **Pick optimizer:** start with **Adam** (LR = 10^{-3}).
6. **Add regularization:** dropout (0.2-0.5) or weight decay (10^{-4}).
7. **Monitor train and validation loss** at each epoch.
8. **Use early stopping** based on validation loss.
9. **Scale up**: batch size, depth, width, data augmentation.

Debugging tips:

- If loss is NaN: LR too high or bad init.
- If loss doesn't decrease: check gradients, try different optimizer.
- If overfitting: more regularization, more data, smaller model.

Lecture 3 Summary

1. **Optimizers:** Adam combines momentum + adaptive LR + bias correction. Default choice.
2. **Initialization:** Xavier for sigmoid/tanh, He for ReLU.
3. **Regularization:** L2 shrinks weights, L1 induces sparsity, dropout breaks co-adaptation.
4. **Batch Normalization:** normalize activations per batch, with learnable γ, β.
5. **UAT:** one hidden layer is enough *in theory*, but depth is **exponentially more efficient** for compositional functions.
6. **GPUs:** parallelism enables training of large models. Use PyTorch `.to(device)`.

Next Lecture: Convolutional Neural Networks

Lecture 4 (Saturday, April 25):

- Motivation: why CNNs for images?
- The convolution operation (mathematical details).
- Pooling and modern alternatives.
- Classic architectures: LeNet, AlexNet, VGG, ResNet.
- Applications: satellite imagery for poverty prediction.

Recommended reading:

- Goodfellow et al., *Deep Learning*, Chapters 7–8.
- Kingma & Ba (2015), *Adam*; Ioffe & Szegedy (2015), *Batch Normalization*.

Homework 1 is now available! Due Sunday, April 27. Covers Lectures 1–3.

Questions?

References

- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *ICLR*.
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods. *JMLR*, 12, 2121–2159.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers. *ICCV*.
- Srivastava, N. et al. (2014). Dropout: A simple way to prevent neural networks from overfitting. *JMLR*.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization. *ICML*.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Math. Control Signals Syst*.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*.